

Crop Disease Detection

A CNN + Flask portfolio project
on the PlantVillage tomato dataset

Nikhil Rana

Design decisions, incident write-ups and anticipated questions

Contents

Interview Prep	2
Phase 1 — Project structure	4
What was built	4
Why — design decisions	4
Key metrics	4
Likely interview questions	5
Phase 2 — Dataset download and EDA	5
What was built	5
Why — design decisions	5
Key metrics / EDA findings	5
Decisions this drives in later phases	6
Likely interview questions	6
Phase 3 — Preprocessing and augmentation pipeline	6
What was built	7
Why — design decisions	7
Key metrics	7
Likely interview questions	8
Phase 4 — Architecture and benchmark	9
What was built	10
Architecture	10
Why — design decisions	10
Benchmark results	10
Likely interview questions	12
The cache experiment — a measured negative result	13
The paging incident — how the cache went from useless to harmful	13
Final training configuration — engineering to the constraint	14
Actual training results	16
Phase 5 — Evaluation	16
What was built	18
Why the test split and not validation	18
Headline results	18
Macro-F1 vs weighted-F1 — and which matters here	18
Best and worst classes — and why	18
Reading the confusion matrix	18
Likely interview questions	19
Phase 6 — Flask app	19

What was built	20
Why — design decisions	20
Error handling	20
Likely interview questions	21
Elevator Pitch	21
Appendix — Anticipated Questions	22
Project structure and tooling	23
Dataset and EDA	23
Preprocessing	23
Architecture and training	23
Evaluation	24
Deployment	24

Interview Prep

A running log of **what was built, why it was built that way, and the questions an interviewer is likely to ask** about it. One section per phase, appended as each phase is completed.

Phase 1 — Project structure

What was built

The empty skeleton of the project — no data, no model, no app code yet, just the scaffolding everything else drops into:

- **Folder tree:** `data/raw`, `data/processed`, `notebooks`, `src`, `models`, `app/templates`, `app/static`.
- `requirements.txt` listing every dependency with a comment explaining why it is there.
- `.gitignore` excluding datasets, trained models, `__pycache__`, the virtual environment and secrets — while keeping the empty folders visible via `.gitkeep` placeholder files.
- `README.md` with the project description, setup instructions, the folder layout, and placeholder sections for Dataset / Model / Results / Usage.

Why — design decisions

Why separate `data/raw` from `data/processed`? `raw` is treated as read-only: it is exactly what was downloaded from Kaggle and never modified. Every transformation writes to `processed`. This means (a) the slow download never has to be repeated, and (b) if a preprocessing step turns out to be buggy, the pipeline can be re-run from a known-good source instead of from data that has already been silently corrupted.

Why keep data and models out of Git? Git stores a complete new copy of a file every time it changes, and it is built for text (where it can store just the differences), not for large binaries. `PlantVillage` is over a gigabyte, and a saved Keras model is tens of megabytes. Committing them would make the repo slow to clone, and GitHub hard-rejects any file over 100 MB. Both are *reproducible* — the dataset from Kaggle and the model from the training script — so committing the code that produces them is enough.

Why `.gitkeep` files? Git tracks *files*, not *folders*, so an empty folder simply does not exist as far as Git is concerned. Someone cloning the repo would get no `data/raw` at all, and the download script would crash. A zero-byte `.gitkeep` file gives Git something to track so the folder survives the clone. (The name is a convention, not a Git feature — any filename would work.)

Why `>=` version ranges instead of exact `==` pins? For a portfolio project that other people will clone onto unknown machines, minimum versions keep installation from breaking over trivial version skew. Production systems do the opposite and pin exactly, so that a deploy today installs precisely what was tested last week. The trade-off is *reproducibility* vs. *flexibility*, and the right answer depends on which one costs more when it goes wrong.

Why Flask rather than Django or FastAPI? The app is two routes: show an upload form, and return a prediction. Django brings an ORM, a migration system and an admin panel that would all sit unused. FastAPI would be the better pick for a pure JSON API, but this app renders an HTML page, which is Flask's natural home.

Why write `INTERVIEW_PREP.md` as the project goes rather than at the end? The reasoning behind a decision is sharpest at the moment it is made. Recording it immediately avoids reconstructing a plausible-sounding rationale weeks later.

Key metrics

None yet — no data or model exists at this phase. First metrics arrive in Phase 2 (dataset statistics) and Phase 4 (training accuracy).

Likely interview questions

Q: Why did you structure the project this way instead of putting everything in one script? A: Separating data, source code, models and the app means each part can change independently. I can rewrite the preprocessing without touching the Flask app, or swap the model file without touching preprocessing. It also makes the project readable to someone who has never seen it: the layout is a trimmed-down version of the standard *Cookiecutter Data Science* template, so the folder names are already familiar. For a single throwaway experiment one script is genuinely fine — the structure earns its keep once there are multiple stages that get re-run at different times.

Q: What would happen if you committed the dataset and the trained model to Git? A: The repo would balloon to over a gigabyte, cloning would take minutes, and GitHub would reject any single file above 100 MB. Worse, because Git snapshots a whole new copy of a binary on every change, retraining the model a few times would permanently bloat the history — and deleting the file later does not shrink it, since the old versions stay in the history. The standard fixes are either to ignore the artefacts and regenerate them (what I did here), or to use Git LFS / DVC, which store large files outside the repo and keep only a pointer in Git.

Q: Why is a virtual environment worth the extra step? A: It gives this project its own isolated copy of its libraries. Without one, `pip install` puts packages system-wide, so a different project needing an older NumPy would break this one. It also makes `requirements.txt` honest — I can be confident the file lists everything needed, because the environment started empty.

Q: You put `.env` and `kaggle.json` in `.gitignore` — why does that matter? A: `kaggle.json` holds an API key tied to my Kaggle account. If it were pushed to a public repo, anyone could use it — and secret-scanning bots find leaked keys within minutes. It also cannot be un-leaked by deleting the file in a later commit, because the key is still readable in the Git history. The only real fix after a leak is to revoke and reissue the key, so the important thing is not to commit it in the first place.

Phase 2 — Dataset download and EDA

What was built

- `src/download_data.py` — a scripted, repeatable download of the PlantVillage archive from Kaggle that extracts **only** the 10 colour tomato classes directly out of the zip.
- `notebooks/01_eda.ipynb` — exploratory analysis covering class distribution, sample images per class, image size/format statistics, and a corrupt-file check.
- **README "Dataset" section** — source, scope, the per-class table, the reasoning for subsetting, and the EDA findings.

Why — design decisions

Why script the download instead of clicking "Download" on Kaggle? A script is reproducible. Anyone cloning the repo runs one command and gets byte-identical data, with no written instructions to misread. It also documents *exactly* which dataset was used — a manual download leaves no record of the version or the source.

Why extract from inside the zip rather than unzipping everything? The archive holds 162,916 files: 38 classes across 14 crops, each stored three times (color / grayscale / segmented). We need 18,160 of them. Streaming the matching members straight out of the zip means the other ~145,000 files are never written to disk.

— a meaningful saving in both time and space on Windows, where creating many small files is slow.

Why the *colour* variant and not grayscale or segmented? Several tomato diseases are distinguished by the *hue* of the lesion — yellow mottling for mosaic virus versus brown concentric rings for early blight — information grayscale throws away. The "segmented" variant has the background removed, which would make the task artificially easier than a real photo and would not match what the Flask app receives at inference time.

Why keep `data/raw` untouched and extract into a subfolder? The extraction writes to `data/raw/tomato/<class>/`, preserving the "folder name = label" convention. Keras' `image_dataset_from_directory` reads exactly that layout in Phase 3, so no separate CSV of labels is needed and there is no risk of labels drifting out of sync with files.

Why check for corrupt files before training rather than during? A truncated JPEG raises an exception the moment the data loader touches it, which kills a training run mid-epoch — potentially 30 minutes in. A five-minute integrity pass up front turns a random future failure into a known quantity. (Result: zero corrupt files, so no guard clause is needed.)

Key metrics / EDA findings

Finding	Value
Total images	18,160
Classes	10 (9 diseases + healthy)
Largest class	Tomato Yellow Leaf Curl Virus — 5,357 (29.5%)
Smallest class	Tomato mosaic virus — 373 (2.1%)
Imbalance ratio	14.4×
Majority-class baseline accuracy	29.5%
Resolution	256×256 for 100% of images
Colour mode / format	RGB / JPEG for 100% of images
Aspect ratios present	1.0 only (all square)
Corrupt / unreadable files	0

Class balance. Imbalance is real but moderate — 14.4× between the extremes, not 1000×. The single most useful number here is the **29.5% majority baseline**: it is what a model scores by ignoring the image entirely, and it is the floor any real result must clear.

Image uniformity. Total consistency (256×256 RGB JPEG, no exceptions) is unusual and convenient: no aspect-ratio distortion, no cropping decisions, no mixed colour modes. It is also a *tell* about provenance — these are controlled lab photographs of detached leaves on plain backgrounds, not field photos.

Decisions this drives in later phases

- **Stratified splitting (Phase 3).** A plain random split could leave the

373-image mosaic-virus class with very few validation images, making its score statistical noise. Stratifying preserves class proportions in every split.

- **Metrics beyond accuracy (Phase 5).** With a 29.5% baseline and a 14.4×

imbalance, headline accuracy hides per-class failure. A confusion matrix and per-class recall are the honest reporting.

- **No resampling yet.** Establish a baseline first, then fix what the confusion

matrix actually shows is broken. Class weights are the obvious lever if the rare classes underperform.

Likely interview questions

Q: Why didn't you use the full 54,000-image dataset? A: Two reasons, one practical and one about problem quality. Practically, I train a CNN from scratch on a CPU, so 18k images versus 54k is roughly a 3× difference in

epoch time — that is what makes it feasible to iterate on the architecture several times rather than once. More interestingly, the full dataset is in some ways an *easier* problem: much of it is telling apples from grapes from corn, which is a crop-identification task the network can solve from overall leaf shape and colour. Restricting to one crop forces the model to discriminate between diseases of the same species — early blight versus late blight versus Septoria leaf spot — which is the genuinely hard, and genuinely useful, part. A 92% on tomato-only means more than 92% across all 38 classes.

Q: You have a 14× class imbalance. How did you handle it? A: In Phase 2 I measured it and let it inform the design rather than immediately "fixing" it. Three concrete responses: **(1)** I quote the 29.5% majority-class baseline alongside any accuracy figure, so nobody reads 85% as impressive without knowing that guessing gets 29.5%. **(2)** The train/val/test split is stratified, so the 373-image mosaic-virus class is proportionally represented everywhere — otherwise its validation score would be noise. **(3)** Evaluation reports a confusion matrix and per-class recall, since aggregate accuracy is dominated by the large classes. I deliberately did *not* resample or reweight for the first run: the point of a baseline is to find out what actually breaks. If mosaic virus shows poor recall in Phase 5, class weights are the first lever — they penalise mistakes on rare classes more heavily without discarding data the way undersampling does, or overfitting to duplicates the way naive oversampling can.

Q: What did the EDA tell you that changed your plan? A: Three things. The image uniformity — every file 256×256 RGB — meant I could drop the aspect-ratio handling I had expected to need; resizing is a clean downscale. Zero corrupt files meant no defensive error handling in the data loader. And the imbalance figure is what pushed me to stratify the split and to plan the evaluation around a confusion matrix rather than accuracy. EDA is cheap and each of those findings removed either work or a future failure.

Q: What is the biggest weakness of this dataset? A: The domain gap. PlantVillage images are lab photographs of single detached leaves on a plain, uniform background. A real user photographs a leaf still on the plant, with soil, sky and other foliage in frame, in uncontrolled lighting. A model trained on this data can score very well on its own test split and still degrade badly on real phone photos, because the test split shares the same unrealistic conditions as the training data. High test accuracy here proves the pipeline works; it does not prove field readiness. Closing that gap would need either field-collected images or aggressive augmentation (random backgrounds, lighting and scale) — and honestly reporting the limitation matters more than hiding it behind a good number.

Phase 3 — Preprocessing and augmentation pipeline

What was built

- `src/data_loader.py` — the full path from folders of JPEGs to batched

tensors: manifest building, stratified 70/15/15 splitting, persisted split CSVs, `tf.data` pipelines (decode → resize 128×128 → scale to [0,1]), and Keras augmentation layers applied to training data only.

- `notebooks/02_pipeline_check.ipynb` — six explicit PASS/FAIL checks run

before any training, covering shape, pixel range, label alignment, stratification, augmentation behaviour, and val/test purity.

- **README "Preprocessing" section** documenting each choice.

Why — design decisions

Why in-graph preprocessing rather than writing processed images to disk? Three reasons, and the first is the decisive one:

- **Augmentation has to be on the fly to work at all.** The point of

augmentation is that the model sees a *differently* distorted copy of each image every epoch. Writing augmented copies to disk freezes a fixed set of variants — the model just memorises 5 versions instead of 1, and most of

the regularisation benefit disappears.

- **No stale derived data.** Changing the input size means changing one

constant. With a preprocessed copy on disk, there is a second dataset that can silently disagree with the code that produced it — and nothing warns you.

- **Cost.** No second 18k-image copy to write, store or keep in sync.

The trade-off is CPU work per epoch: decoding and resizing happen every time rather than once. `prefetch` and parallel decoding hide most of that cost, and for an 18k-image dataset it is clearly the right side of the trade. At a scale where decoding genuinely bottlenecks the model, the answer is `dataset.cache()` or a pre-built `TfRecord` — not a folder of JPEGs.

Why persist the split manifest, when nothing else is persisted? Because the split must *never* change, while everything else must be free to. If the split were recomputed with a different shuffle between training and evaluation, images the model trained on would land in the test set and inflate the score. Saving three CSVs of paths (a few hundred KB) fixes the split permanently, and a fixed `random_state=42` makes it reproducible from scratch.

Why stratify? With a 14.4× imbalance, a plain random split could deal the 373-image mosaic-virus class an unlucky hand — leaving it with, say, 30 test images, where a handful of errors swings its recall by 10 points and the number means nothing. Stratifying forces every split to mirror the overall distribution. Measured result: the largest class-share drift between any two splits is **0.1 percentage points**.

Why these three augmentations specifically? Each models a variation that genuinely occurs when photographing a leaf, and none of them changes the correct answer:

- `RandomFlip` (horizontal + vertical) — a leaf has no inherent "up"; a flipped

leaf is the same disease. Free, completely safe variety.

- `RandomRotation(0.1)` ($\approx \pm 36^\circ$) — the camera not held square to the leaf.
- `RandomZoom(0.1)` — the camera slightly nearer or further.

Why not more aggressive augmentation? This is the more interesting half, and it is specific to this domain:

- **No colour/brightness/hue jitter.** Diagnosis here depends on hue — yellow

mottling means mosaic virus, brown concentric rings mean early blight. Shifting colours attacks the exact signal the model needs, and can push an image towards the appearance of a *different* disease while keeping its original label. That is actively harmful: it teaches the model something false.

- **No shear or perspective warping.** These are flat, square-on lab photographs

of detached leaves. Simulating extreme perspective invents a distribution that exists in neither the training nor the test data, spending model capacity on variation that never occurs.

- **No random cropping.** A lesion can be anywhere on the leaf. A crop may remove the only diseased region while keeping the "diseased" label — again, a wrong-label image.

- **Rotation/zoom capped at $\pm 10\%$** because larger rotations pad the corners with empty pixels, and a network will happily learn to read that padding artefact instead of the leaf.

The general principle: augment along axes that genuinely vary (orientation, distance) and leave alone the axes that carry the diagnostic signal (colour).

Key metrics

Split	Images	Share	Imbalance ratio
Train	12,712	70.0%	14.4×
Validation	2,724	15.0%	14.3×
Test	2,724	15.0%	14.4×

Stratification verification — largest class-share drift between any two splits: 0.1 percentage points.

Per-class shares match the full-dataset distribution across all three splits (e.g. Yellow Leaf Curl Virus 29.5% / 29.5% / 29.5%; mosaic virus 2.1% / 2.1% / 2.1%).

Pipeline check results — all six PASS:

Check	Result
Batch shape	(32, 128, 128, 3) float32 for all splits
Pixel range	min 0.000, max ≈0.95–0.97 → normalised exactly once
Label validity	indices 0–9, all 10 classes seen across 320 samples
Augmentation alters images	mean abs. diff from original 0.121
Augmentation is random	mean abs. diff between two calls 0.094
Val/test unaugmented	two passes bit-for-bit identical (max diff 0.0000000000)

Likely interview questions

Q: Why do you augment only the training data and not validation or test? A: Because they answer different questions. Training data exists to teach the model, so distorting it is useful — it forces the model to learn the underlying pattern rather than memorise specific photographs. Validation and test data exist to *measure* the model on realistic inputs. If I augmented them, I would be measuring performance on randomly distorted images that no user will ever submit, and worse, the score would jitter between runs as the random distortions changed — so I could not tell whether a change to the model helped or the dice just fell differently. There is a subtle trap here too: Keras preprocessing layers are deliberately no-ops when called with `training=False`, so if you put augmentation *inside* the model rather than in the input pipeline, it automatically disables itself at evaluation time. I kept it in the `tf.data` pipeline and applied it explicitly with `training=True`, then wrote a test asserting that two passes over the validation set are bit-for-bit identical — I would rather assert it than assume it.

Q: Why those augmentation types and not others? A: The rule I used is that an augmentation must change the image without ever changing the correct label, and it should model a variation that actually occurs. Flips, small rotations and small zooms all pass: a leaf photographed upside down or from slightly further away is still the same disease. Colour jitter fails, and that is domain-specific reasoning — several of these diseases are identified *by* colour, so shifting hue can make a mosaic-virus leaf look like early blight while the label still says mosaic virus. That does not regularise the model, it poisons the labels. Random cropping fails for the same reason: crop out the only lesion and you have a picture of a healthy leaf labelled "diseased". This is the part people get wrong by reaching for a standard augmentation recipe — the right set depends entirely on what carries the signal in your data.

Q: How do you prevent data leakage between splits? A: Four things. First, the split is on *file paths*, made once, before any image is loaded — so a single physical image can only ever land in one split. Second, the split is written to CSV and read back, rather than recomputed; a fresh shuffle between training and evaluation is the classic way test images silently end up in training. Third, `random_state` is fixed, so the split is identical on every machine and every run. Fourth — the one people forget — **all preprocessing statistics must come from training data only**. Here that is trivially satisfied because normalisation is a fixed divide by 255, not a mean/std computed from the data. If I had used per-channel standardisation, I would have had to compute the mean and standard deviation on the training split alone and apply those same constants to val and test; computing them over the whole dataset would leak information about the test set into the model's inputs.

Q: Why 128×128 rather than keeping the native 256×256? A: Compute. Halving each dimension quarters the pixel count, so roughly quarters the convolution work per image. Training from scratch on a CPU, that is the difference between iterating on the architecture several times in an evening and managing one run overnight —

and the number of experiments you can afford is usually what determines the final result. I did not go smaller than 128 because at 96×96 the fine speckling separating Septoria leaf spot from Target Spot starts to smear, and no model can learn a feature that is no longer present in its input. If accuracy on those two classes turns out to be the weak point in Phase 5, raising the input size is one of the first things I would try.

Phase 4 (part 1) — Architecture and benchmark

Training results follow after the full run; this section covers the design and the timed benchmark only.

What was built

- `src/model.py` — `build_model()` returning the compiled CNN: three

Conv→BatchNorm→MaxPool blocks (32/64/128 filters), GlobalAveragePooling, Dense(128) + Dropout(0.5), Dense(10, softmax). **111,946 parameters.**

- `src/train.py` — loads the Phase 3 pipelines, computes inverse-frequency

class weights from the training split, and runs either a **one-epoch timed benchmark** (default) or the full run with early stopping (`--full`).

- **Split manifests committed.** `data/processed/*.csv` is now un-ignored, so the exact split is reproducible from the repository rather than from a seed.

Architecture

Layer	Output shape	Params
Input	(128, 128, 3)	0
conv1 — Conv2D(32, 3×3, relu, same)	(128, 128, 32)	896
bn1 — BatchNormalization	(128, 128, 32)	128
pool1 — MaxPooling2D(2×2)	(64, 64, 32)	0
conv2 — Conv2D(64, 3×3, relu, same)	(64, 64, 64)	18,496
bn2 — BatchNormalization	(64, 64, 64)	256
pool2 — MaxPooling2D(2×2)	(32, 32, 64)	0
conv3 — Conv2D(128, 3×3, relu, same)	(32, 32, 128)	73,856
bn3 — BatchNormalization	(32, 32, 128)	512
pool3 — MaxPooling2D(2×2)	(16, 16, 128)	0
gap — GlobalAveragePooling2D	(128)	0
dense1 — Dense(128, relu)	(128)	16,512
dropout — Dropout(0.5)	(128)	0
output — Dense(10, softmax)	(10)	1,290
Total		111,946

Trainable 111,498 · non-trainable 448 (BatchNorm moving averages).

Why — design decisions

Why three conv blocks, not two or five? Each block halves the spatial size: 128 → 64 → 32 → 16. Three blocks land on a 16×16 map, which is the sweet spot — small enough that global pooling summarises it meaningfully, large enough that spatial information has not been thrown away. Two blocks would stop at 32×32 with only 64 filters, leaving the deepest features still fairly local — they would describe textures, not lesion-scale structure. Five blocks would take it to 4×4, which for 128×128 inputs starts discarding detail, and would add parameters to a model that already has to be regularised hard against 12,712 training images. Depth is also not

free here: the benchmark says 8.6 minutes per epoch on CPU, and a fourth block would add meaningfully to that. Three is the point where the receptive field covers a useful fraction of the leaf without either wasting compute or overfitting.

Why 32 → 64 → 128 filters? Doubling the filter count each time the spatial size halves is the standard CNN trade-off, and it is a genuine trade rather than a convention. Early layers see small patches and only need a few kinds of pattern — edges, colour transitions — so 32 filters suffice. Deeper layers see larger regions, and the number of *distinct* things a large region can contain is much greater, so more filters are needed to describe them. Doubling filters while quartering the pixel count also keeps the computational cost per block roughly balanced rather than front-loading or back-loading the network. Starting at 32 rather than 64 halves the cost of the most expensive layer — conv1 runs at full 128×128 resolution.

Why GlobalAveragePooling2D instead of Flatten? This is the single highest-leverage choice in the architecture. The final feature map is 16×16×128. Flatten would produce a 32,768-element vector, and connecting that to Dense(128) costs $32,768 \times 128 \approx 4.2$ million parameters — turning a 112k-parameter model into a 4.3M-parameter one, where **97% of the weights sit in a single layer**. With 12,712 training images that layer would overfit almost immediately; it has enough capacity to memorise the training set outright.

GlobalAveragePooling2D instead averages each of the 128 feature maps down to one number, giving a 128-element vector and a 16,512-parameter dense layer — a **254× reduction**. It also gives something Flatten cannot: translation invariance. The average of a feature map does not depend on *where* in the leaf the feature appeared, which is exactly right here, because a lesion in the top left and the same lesion in the bottom right mean the same diagnosis. Flatten would force the model to learn that equivalence from data instead.

Why BatchNorm after each Conv? As a network trains, the distribution of each layer's inputs keeps shifting because the layers below it are still changing. BatchNorm renormalises each batch's activations back to a stable scale, which (a) lets training use a higher learning rate without diverging, (b) speeds up convergence — worth a lot when an epoch costs 8.6 minutes — and (c) provides a mild regularising effect, since each example's normalisation depends on the other examples in its batch.

Worth knowing for follow-up questions: I placed BatchNorm *after* the ReLU activation (Conv(reLU) → BN → Pool). The original paper puts it *before* the activation (Conv → BN → ReLU), and that ordering is marginally more common in modern code. In practice the difference is small and empirically contested; the important thing is to know that both exist and that the choice is deliberate.

Why Dropout 0.5 on the dense layer only, and not after the conv layers? Dropout randomly zeroes units during training so the model cannot lean on any single feature. It belongs where the parameters are, and after global pooling the dense layers hold 16,512 + 1,290 of the weights in the most overfit-prone part of the network. 0.5 is the rate the original Dropout paper recommends for fully connected layers and is a sensible default.

Conv layers are a different case. They already have far fewer parameters (weight sharing means one 3×3 filter is 9 weights regardless of image size), and they are already regularised by BatchNorm, by pooling, and by the input augmentation from Phase 3. Dropping random pixels from a feature map is also less effective than it sounds, because neighbouring pixels are highly correlated — the dropped information is usually still present next door. If the model still overfits, SpatialDropout2D (which drops entire feature maps) is the right tool, not plain Dropout.

Why class_weight rather than oversampling or focal loss? All three address the 14.4× imbalance; they differ in cost and side-effects.

- `class_weight` (**chosen**) multiplies each example's contribution to the loss

by its class weight, so every *class* contributes roughly equally rather than every *image*. It costs nothing — no extra data, no change to epoch time — and it is a single argument to `fit()`, so it is trivial to ablate: run with and without, compare. That matters when each experiment costs 8.6 min/epoch.

- **Oversampling** duplicates rare-class images. The 373-image mosaic-virus class

would need ~14× duplication, and the model then sees the *same* 373 images over and over, which invites memorising them specifically. It also lengthens every epoch by inflating the dataset.

- **Focal loss** down-weights easy examples to focus on hard ones. It is designed

for extreme imbalance (object detection, where background outnumbered objects ~1000:1) and introduces a second hyperparameter, γ , to tune. At 14.4× it is reaching for a heavier tool than the problem needs.

The honest framing: at moderate imbalance, class weighting gets most of the benefit for none of the cost. If Phase 5's confusion matrix shows mosaic virus still failing, focal loss or targeted data collection become worth the extra complexity.

Computed weights (from the training split only, so nothing leaks from val/test):

Class	Train images	Share	Weight
Tomato Yellow Leaf Curl Virus	3,750	29.5%	0.339
Bacterial spot	1,489	11.7%	0.854
Late blight	1,336	10.5%	0.951
Septoria leaf spot	1,240	9.8%	1.025
Spider mites (two-spotted)	1,173	9.2%	1.084
Healthy	1,114	8.8%	1.141
Target spot	983	7.7%	1.293
Early blight	700	5.5%	1.816
Leaf mold	666	5.2%	1.909
Tomato mosaic virus	261	2.1%	4.870

Weight ratio $0.339 \rightarrow 4.870 = 14.4\times$, exactly mirroring the class imbalance, which is the arithmetic working as intended: `weight = n_samples / (n_classes * class_count)`, so a class holding exactly 1/10th of the data gets weight 1.0.

Why early-stop on `val_loss` rather than `val_accuracy`? Because with a 14.4× imbalance, accuracy is a blunt instrument. A model can raise accuracy simply by getting better at Yellow Leaf Curl Virus — 29.5% of the data — while getting worse at mosaic virus at 2.1%. Accuracy would go up and the model would be worse at the thing that is hard.

Loss is more sensitive in two ways that matter here. It is computed on the predicted *probabilities*, not just the arg-max, so it registers a model becoming less confident about correct answers — a genuine early warning of overfitting that accuracy is blind to, because the arg-max does not change until confidence has already collapsed. And because `class_weight` is applied to the loss, the monitored quantity is the *class-balanced* loss, which is exactly the objective being optimised. Accuracy is also a step function of the arg-max, so it moves in discrete jumps and plateaus — poor signal for a patience-based rule, which needs to detect small consistent degradation.

Benchmark results (one timed epoch)

Measurement	Value
Time for 1 epoch	514 s (8.6 min) — 398 steps at ~1.3 s/step
Estimated 30 epochs	~4.3 hours (conservative — epoch 1 includes graph tracing)
Training loss / accuracy	1.176 / 61.21%
Validation loss / accuracy	3.794 / 23.94%
Random-guess loss, $\ln(10)$	2.303
Majority-class baseline	29.5%

Reading these numbers. Training loss of 1.176 against a random baseline of 2.303, with 61% training accuracy after a single pass, says the model is learning real structure — this is the check the benchmark exists for.

The validation numbers are *worse* than random-ish, and that is worth explaining rather than glossing over. Validation loss (3.79) far above training loss (1.18) after one epoch is the classic **BatchNorm warm-up** signature. During training, BatchNorm normalises using each batch's own statistics; at inference it uses moving averages accumulated across training. After one epoch those averages have only had 398 updates at momentum 0.99 and are still far from the true activation statistics, so the model behaves quite differently in inference mode than in training mode. This typically resolves within a few epochs as the moving averages converge. It is a known artefact of measuring after exactly one epoch, not evidence that the architecture is broken — but the way to *confirm* that is the full run, not assertion.

Likely interview questions

Q: Your model has 112k parameters. Isn't that far too small to be useful? A: It is small deliberately, and the constraint that sets the size is the data, not the ambition. There are 12,712 training images; a model with millions of free parameters can memorise that outright, and the validation curve would diverge from training within a few epochs. The parameter count is also where the GlobalAveragePooling choice pays off — swapping it for Flatten would take the model from 112k to 4.3M parameters, with 97% of them in one dense layer, purely to preserve spatial position information the task does not need. Small also means fast: 8.6 minutes per epoch on CPU, so the whole run fits in an afternoon and I can afford to try things. If the model turns out to *underfit* — training accuracy plateauing well below what the task allows — then adding capacity is the right response, and I would add a fourth conv block before I would touch the dense head.

Q: You measured validation loss of 3.79 after one epoch, far worse than training loss of 1.18. Isn't that overfitting already? A: It is very unlikely to be overfitting after a single pass through the data — a 112k-parameter model has not had the opportunity to memorise 12,712 images in 398 gradient steps. The much more probable cause is BatchNorm's train/inference mismatch. In training mode BatchNorm normalises using the current batch's mean and variance; in inference mode it uses moving averages accumulated during training, which after one epoch are still poorly estimated. So the training number is measured with good statistics and the validation number with bad ones, and they are not yet comparable. The distinguishing test is simply to keep training: BatchNorm warm-up closes over the next few epochs, whereas genuine overfitting shows validation loss *rising* while training loss keeps falling, after both have first come down together. If the gap persisted past five or six epochs I would suspect something structural — and I would check the augmentation was not accidentally applied to validation, which Phase 3 already tests for.

The cache experiment — a measured negative result

After the first benchmark showed 8.6 min/epoch, the obvious suspicion was that decoding 12,712 JPEGs on every epoch was wasted work. So I added a `tf.data` cache and measured. **It produced no speedup.** The result is kept here in full because a profiled negative result is worth more than an unmeasured assumption.

What was cached, and where in the pipeline. The cache sits immediately after decode + resize, which is the expensive stage whose output never changes:

read → decode → resize 128×128	expensive, deterministic
[normalise]	val/test only
CACHE	← everything above runs once
shuffle	must be AFTER cache
batch	
[normalise]	train only
augment	must be AFTER cache
prefetch	

Why shuffle and augment must sit after the cache. `cache()` records what a stage produces and replays it identically forever. Anything random placed *before* it gets frozen: a shuffle before the cache yields one fixed ordering for the entire run, and augmentation before the cache yields one fixed set of distortions — which would silently destroy most of augmentation's value while still looking like it was working. This is the subtle trap in `tf.data` caching, and it is invisible unless you deliberately test for it. The Phase 3 checks (train differs between passes, val identical) are exactly that test.

Why uint8 and not float32. The cache stores the resized image *before* normalisation, so pixels stay one byte per channel instead of four:

	per image	× 12,712 train images
uint8 (128×128×3)	49,152 B	~596 MB
float32 (128×128×3)	196,608 B	~2,384 MB

A 4× saving for identical benefit, because the divide-by-255 is arithmetically trivial next to JPEG decoding — there is no reason to spend four bytes storing what one byte plus a cheap multiply reproduces. Measured on-disk cache came out at **595.9 MB**, matching the prediction.

Validation and test are the opposite case and *are* cached as float32 (**510.8 MB** measured): they are never augmented and never shuffled, so their batches are byte-identical every epoch, and caching post-normalisation means zero per-epoch work rather than a cheap-but-nonzero multiply.

Why disk-backed rather than in-memory. The machine has 7.3 GB of RAM with only **~491 MB genuinely available**. Training needs ~596 MB (train, uint8) plus ~511 MB (val, float32) ≈ **1.1 GB of cache**. An in-memory cache that does not fit does not fail loudly — the OS pages it to disk, giving a disk cache with extra copying on top, which is strictly worse than asking `tf.data` to use the disk deliberately. Measuring available RAM *before* choosing the backend turned a likely thrashing run into a clean one.

The result:

Epoch	Time	
1 — fills cache, full JPEG decode	504.0 s	8.4 min
2 — reads from cache	518.8 s	8.6 min
Speedup	0.97×	<i>slightly slower</i>

Why it didn't help: the workload is compute-bound, not I/O-bound. Counting multiply-accumulates in the forward pass, per image:

Layer	Output	MACs
conv1	128×128×32, kernel 3×3×3	$16,384 \times 32 \times 27 \approx \mathbf{14.2\ M}$
conv2	64×64×64, kernel 3×3×32	$4,096 \times 64 \times 288 \approx \mathbf{75.5\ M}$
conv3	32×32×128, kernel 3×3×64	$1,024 \times 128 \times 576 \approx \mathbf{75.5\ M}$
		≈ 165 M MACs / image

Backpropagation costs roughly twice the forward pass, so ≈ 495 M MACs per image per step, × 12,712 images ≈ **6.3 TMACs per epoch**. At 504 s that is ~12.5 GMAC/s sustained on an **AMD Ryzen 5 5500U (6 cores / 12 threads, 2.1 GHz base)** — entirely consistent with a mobile CPU running oneDNN convolutions. The arithmetic accounts for essentially the whole epoch time, leaving no room for I/O to have been the constraint.

In other words, the Phase 3 pipeline was *already* keeping the model fed: `num_parallel_calls=AUTOTUNE` decoded images across cores while `prefetch` overlapped that with compute, so decoding was fully hidden behind the model's own arithmetic. Caching removed work that was not on the critical path.

Why the code was kept anyway. It is correct, it is off by one argument (`cache=False`), and on a machine where the model is cheaper or the storage slower it would help. It also came with a verification harness proving the cached pipeline is bit-identical to the uncached one (max abs diff 0.0000000000 on val and test), which is worth having regardless. The honest summary is: *the optimisation was measured, it did not help on this hardware, and the measurement explains why*.

What would actually speed this up, in order of expected effect: reducing the input size (halving to 96×96 roughly halves the MACs, at a real cost to the fine texture that separates Septoria leaf spot from Target Spot); giving conv1 a stride of 2 so the most expensive layer runs at half resolution; or simply using a GPU, where this model would train in minutes. None of those are free, which is the point — the cache looked free, and that is precisely why it was worth measuring rather than assuming.

The paging incident — how the cache went from useless to harmful

The cache did not merely fail to help. It **broke the next training run**, and the mechanism is the most instructive thing in this project.

What happened. The 30-epoch run was launched expecting ~4.3 hours. After **5 hours 21 minutes it had completed roughly 11 of 30 epochs** and was still slowing down. It was killed deliberately; the projected remaining time had grown past 10 hours.

The trigger: a buffer that changed size without changing its code. Caching requires `shuffle` to move *after* the cache — otherwise `cache()` records one fixed ordering and replays it every epoch, silently destroying the shuffle. That reordering was correct and necessary. What it also did was change what the shuffle buffer *contains*:

	Position	Buffer holds	Size
Phase 3 (uncached)	shuffle before decode	12,712 file path strings	~1 MB
Phase 4 (cached)	shuffle after decode	12,712 decoded images	~596 MB

The line `ds.shuffle(len(paths), ...)` was never edited. Its memory footprint grew **600-fold** purely because of what now sat above it in the pipeline. I had sized the *cache* carefully against available RAM (596 MB train + 511 MB val) and completely failed to re-examine the buffer that the same edit had inflated.

How it manifested. Not as a crash or an error — as a slow strangulation. Measuring CPU-seconds consumed per wall-second (how many cores' worth of work the process was actually getting on a 6-core/12-thread CPU):

Window	CPU-s total	Parallelism
0 → 11 min	4,679	7.36× (healthy)
11 → 126 min	15,191	1.52×
126 → 144 min	22,586	6.73×
144 → 310 min	37,641	1.51×

At the kill: **2,679 MB of process pagefile usage**, ~1,066 MB system RAM available, sustained non-zero Pages/sec. CPU-seconds *per epoch* stayed roughly constant at ~3,700 — the work never changed, the machine simply could not deliver it. The process was waiting on disk, not computing.

Why the benchmark missed it. The 2-epoch benchmark reported 504.0 s and 518.8 s per epoch, consistent with the 514 s uncached baseline. It looked clean. Two reasons it was blind:

- **Too short.** Degradation was progressive, not immediate. Two epochs never

let memory pressure compound.

- **Different conditions.** Available RAM on a 7.3 GB laptop varies with

whatever else is open. The benchmark ran when the buffer still fit; the overnight run did not.

What was recovered. `ModelCheckpoint(save_best_only=True)` had been writing throughout, so the best weights survived: **val_loss 0.3485, val_accuracy 87.67%** (evaluated after the fact), against a 29.5% majority baseline. The model was training *well*. The failure was infrastructure throughput, not architecture, data, or optimisation — an important distinction, because the two look identical from a stalled progress bar.

What was lost. `training_history.json` and `training_log.txt` are written when `fit()` returns, so the per-epoch history died with the process. The lesson is to stream metrics to disk per epoch (a `CSVLogger`-style callback) rather than serialising everything at the end — checkpoints survived precisely because they were written incrementally.

The fix.

- **Bound the shuffle buffer explicitly** at 2,048 images (~94 MB) rather than

`len(dataset)`. Sufficient because the stratified split already randomised the manifest order, so the buffer only needs to add per-epoch variation on top of an already-shuffled base — not perform the whole shuffle itself.

- **Disable the cache by default.** Two independent reasons, either sufficient

alone: it delivered 0.97× (no benefit), and it was the change that forced the shuffle reordering in the first place.

The general lesson — the one worth carrying to any pipeline. *Reordering operations in a data pipeline silently changes the memory footprint of the operations around them.* Every stage's cost depends on what it receives, and moving one stage rewrites the inputs of its neighbours. A code review of that diff would show `shuffle` moving three lines down and conclude "ordering fix, no behaviour change" — while the actual change was a 600× memory increase in a line nobody touched.

Two corollaries:

- ****Benchmarks must run long enough for the failure mode you have not thought**

of.** A 2-epoch benchmark measures steady-state compute. It cannot measure resource exhaustion, which is by definition cumulative. If a run is going to last hours, some part of the validation has to last long enough for pressure to build.

- **Watch the resource, not just the clock.** Wall-time per epoch told me nothing

until it was hours too late. CPU-seconds per wall-second identified the problem immediately and unambiguously: a compute-bound process getting 1.5 cores out of 6 is not slow, it is *blocked*.

Final training configuration — engineering to the constraint

Two training runs were killed before completing: the first (128×128, batch 32, 30 epochs) reached ~11 of 30 epochs in 5 h 21 min; the second, after the shuffle fix, reached 1 of 15 in 156 minutes at **0.91× CPU parallelism**. Both died to the same underlying cause — a 7.5 GB laptop carrying 18–21 GB of commit charge had no memory left to give.

At that point the honest engineering decision is to stop trying to make the machine do something it cannot, and change the shape of the problem:

	Original	Final	Effect
Input size	128×128	64×64	4× fewer pixels, ~4× fewer MACs
Batch size	32	16	half the activation memory per step
Epoch cap	30	5	bounded total runtime
Cache	disk	off	no 1.1 GB of cache I/O
Shuffle buffer	<code>len(dataset)</code>	2,048	~596 MB → ~24 MB
RAM guard	none	≥700 MB at startup	fails fast instead of at hour three

Result: the run completed in 14.5 minutes at 5.4–6.2× CPU parallelism — the healthy range — versus 0.91× when it was thrashing.

What this trade actually costs. At 64×64 the fine speckling that separates Septoria leaf spot from Target Spot is largely destroyed. Those classes should be expected to confuse more than they would at 128×128. That is a deliberate accuracy-for-feasibility trade, and the right way to present it is as a documented constraint with a named remedy (retrain at 128×128 on a GPU), not as a silent choice.

The RAM guard is the durable lesson. `train.py` now measures available memory at startup and refuses to launch below 700 MB. Two runs were lost to a condition that was measurable *before* either started — the information was there, nothing was checking it. Cheap preconditions on expensive operations are almost always worth writing.

Actual training results

Epochs run	5 of 5 (cap reached; early stopping did not fire)
Best epoch	5 — the last one
Total wall-clock	14.5 min (158–207 s/epoch)
Final train	loss 0.4605 , accuracy 84.75%
Final validation	loss 0.7525 , accuracy 75.11%
EarlyStopping	not triggered
ReduceLROnPlateau	not triggered — LR stayed at 1e-3 throughout

Per-epoch:

Epoch	Train loss	Train acc	Val loss	Val acc
1	1.2301	58.89%	2.9081	27.46%
2	0.7849	73.40%	2.8879	38.73%
3	0.6474	78.29%	2.9643	46.04%
4	0.5307	82.38%	0.8263	75.18%
5	0.4605	84.75%	0.7525	75.11%

The BatchNorm warm-up prediction was confirmed. For three epochs validation loss sat near 2.9 while training loss fell steadily from 1.23 to 0.65 — exactly the train/inference mismatch predicted after the very first benchmark. Then at epoch 4 it collapsed from 2.96 to 0.83 and validation accuracy jumped from 46% to 75%. That is not a gradual convergence curve; it is the moment BatchNorm's moving averages became good enough for inference mode to match training mode. Anyone who had killed this run at epoch 3 would have concluded the model was broken.

The run ended because it hit the cap, not because it converged. Validation loss was still falling at epoch 5 (0.826 → 0.753) and early stopping never fired. The model is **undertrained**, and its reported numbers are a floor rather than a ceiling. Saying so is more useful than presenting 5 epochs as if it were a converged result.

Q: Why run a one-epoch benchmark at all instead of just training? A: Two reasons, and the second is the one people underrate. First, it converts an unknown into a number: 8.6 minutes per epoch means 30 epochs is 4.3 hours, which is a decision I can now make deliberately instead of discovering three hours in. Second, it is the cheapest possible smoke test for a broken pipeline. If labels were misaligned or the learning rate were wildly wrong, training loss would sit near $\ln(10) = 2.30$ and I would know within nine minutes rather than at the end of a long run. Getting 1.176 tells me the whole chain — manifest, split, decode, resize, normalise, label mapping, loss function — is wired up correctly. That is a lot of things confirmed for very little time.

Q: You had a 5-hour training run that had to be killed. Walk me through what went wrong. A: I added a `tf.data` cache to speed up training, which required moving the shuffle after the decode step — otherwise the cache would freeze a single fixed ordering and silently destroy the shuffle. That reordering was correct, but it changed what the shuffle buffer *held*: previously 12,712 file path strings (about 1 MB), now 12,712 decoded images (about 596 MB). I had carefully sized the cache against available RAM and never re-examined the buffer, because the line `ds.shuffle(len(paths))` hadn't changed — only what flowed into it had. On a 7.5 GB laptop the run paged itself to a standstill: CPU parallelism collapsed from 7.4× to 1.5×, the process accumulated 2.7 GB of pagefile, and after 5 hours 21 minutes it had finished roughly 11 of 30 epochs and was still slowing, so I killed it. What made it diagnosable was measuring **CPU-seconds per wall-second** rather than wall-time per epoch — a compute-bound process getting 1.5 cores out of 6 is not slow, it's blocked, and that distinction pointed straight at memory rather than at the model. Three things came out of it: I bounded the shuffle buffer explicitly at 2,048 images, I dropped the cache entirely (profiling had already shown it gave a 0.97× "speedup" — the workload was compute-bound, so it was never going to help), and I added a startup RAM guard so `train.py` refuses to launch below 700 MB free rather than discovering the problem three hours in. The broader lesson is the one I'd lead with: **reordering stages in a data pipeline silently changes the memory footprint of the stages around them**, and a code review of that diff would have read "ordering fix, no behaviour change".

Phase 5 — Evaluation

What was built

- `src/evaluate.py` — loads `models/best_model.keras`, runs it over the test

split, and produces overall metrics, a per-class `classification_report`, both confusion matrices, a grid of the most confident errors, and a machine-readable `models/evaluation_metrics.json`.

- `notebooks/04_evaluation.ipynb` — the same results with commentary on what

each one means.

- **README "Results"** — headline metrics, per-class table, confusion matrix.

Why the test split and not validation

Validation was used *during* training: `EarlyStopping` watched it, `ReduceLROnPlateau` watched it, and `ModelCheckpoint` selected which weights to keep based on it. That makes validation performance **optimistically biased** — the model was, in a small but real sense, chosen to do well on it.

The test split was carved out in Phase 3, written to CSV, committed, and read by nothing until evaluation. That is what makes it an honest estimate. This is also why the split had to be persisted rather than recomputed: a reshuffle between training and evaluation would have leaked training images into the test set and inflated every number below.

Headline results

Metric	Value
Test accuracy	76.76%
Test loss	0.7188
Macro F1	0.7448
Weighted F1	0.7732
Majority-class baseline	29.5%
Lift over baseline	+47.3 points

Macro-F1 vs weighted-F1 — and which matters here

Both average per-class F1 scores; they differ in weighting.

- **Weighted-F1** weights each class by its support, so the big classes dominate

and it tracks overall accuracy closely.

- **Macro-F1** is an unweighted mean — the 56-image mosaic-virus test set counts

exactly as much as the 804-image Yellow Leaf Curl set.

Macro-F1 is the more honest number for an imbalanced dataset, because a model can post a strong weighted-F1 while quietly failing on the rare classes. The gap here is **+0.0283** (0.7732 vs 0.7448) — small, which is itself the result: performance is fairly even across classes rather than propped up by the large ones. Had class weighting been omitted, I would expect that gap to be much wider.

Best and worst classes — and why

	Class	F1	Precision	Recall	Support
Best	Tomato mosaic virus	0.913	1.00	0.84	56
Best	Yellow Leaf Curl Virus	0.910	0.95	0.87	804
Worst	Early blight	0.500	0.39	0.70	150
Worst	Target Spot	0.550	0.98	0.38	211

The most interesting result is that mosaic virus — the rarest class, 373 images total and only 56 in test — scores the joint-highest F1. The naive prediction was the opposite: the smallest class should suffer most under a 14.4× imbalance. Two reasons it did not:

- **Class weighting worked.** Mosaic virus carried weight 4.87 versus Yellow

Leaf Curl's 0.339 — a 14.4× multiplier on its contribution to the loss, which is exactly what stopped it being drowned out.

- **It is visually distinctive.** Mosaic virus produces a characteristic yellow

mottling unlike any other class here. Distinctiveness matters more than frequency: a rare but unmistakable class is easier than a common but ambiguous one. The correlation between class size and F1 is only **+0.308** — real but weak, which says visual separability explains more of the variance than sample count does.

The genuine failures are Early blight and Target Spot, and they fail as a *pair*: 24.6% of Target Spot is predicted as Early blight. Both present as brown lesions on leaf tissue, distinguished largely by fine texture — concentric rings versus a more diffuse spot. **That texture is exactly what 64×64 downsampling destroys.** This failure was predicted in Phase 3 when the input size was chosen, and it showed up precisely where predicted. It is a resolution problem, not a modelling one.

Reading the confusion matrix

Bacterial spot is a "sink" class: recall 0.99 but precision 0.54. It catches almost everything genuinely bacterial spot, but also absorbs misclassifications from everywhere else — 90 Yellow Leaf Curl images, 52 healthy, 45 Late blight. A class with high recall and low precision is one the model retreats to when uncertain.

Target Spot is its exact mirror: precision 0.98, recall 0.38. When it commits, it is almost always right; but it misses 62% of them, mostly to Early blight.

The costly direction is disease → healthy, because that means an untreated crop. Here the error runs the *safe* way: 21.8% of healthy leaves are called Bacterial spot (a false alarm costing an unnecessary inspection), while very little flows from disease into healthy. Healthy has **precision 1.00** — the model never wrongly declares a diseased leaf healthy in this test set. For a diagnostic tool, that is the right asymmetry to have, though here it is a fortunate property rather than something explicitly optimised for.

Likely interview questions

Q: Your accuracy is 76.76%. Is that good? A: It depends entirely on the baseline, which is why I always quote it. A model that ignores the image and always guesses the largest class scores 29.5% on this data, so the model is adding about 47 points over guessing. But I would not present 76.76% as this architecture's ceiling — the run stopped at a 5-epoch cap with validation loss still falling and early stopping never firing, on 64×64 inputs chosen because larger configurations would not complete on the hardware I had. It is a floor produced under a documented constraint. The honest comparison class is what published PlantVillage models achieve — high 90s — and the gap is explained by input resolution, training length and the absence of transfer learning, all of which I can name specifically rather than hand-wave.

Q: Why report macro-F1 as well as accuracy? A: Because accuracy hides per-class failure when classes are imbalanced. With a 14.4× imbalance, a model could ignore the rarest classes entirely and still post a respectable accuracy — the big classes carry the average. Macro-F1 gives every class equal vote, so failure on a rare class actually shows up. Here the two are close (0.745 macro vs 0.773 weighted), and that closeness is a

result: it says the class weighting worked and performance is even. If macro-F1 had been, say, 0.55 against a 0.77 weighted, that gap would be the headline finding, not a footnote.

Q: How do you know you haven't leaked test data into training? A: Four structural guarantees. The split is on file paths and made once, before any image is loaded, so one physical image can only land in one split. The split is written to CSV and committed, so it cannot be silently regenerated with a different shuffle between training and evaluation. `random_state` is fixed. And all preprocessing statistics are constants — normalisation is a fixed divide by 255, not a mean computed from data — so there is no channel through which test statistics could influence training. The subtler leak I did guard against is *selection* leakage: validation was used for early stopping and checkpoint selection, which biases it, so I report on test and not validation.

Q: The model over-predicts Bacterial spot. What would you do about it? A: First understand it: recall 0.99 with precision 0.54 means Bacterial spot is where the model retreats when uncertain, absorbing errors from several other classes. Three things I would try, cheapest first. **(1)** Train longer — the run hit its epoch cap while still improving, and an undertrained model defaults to broad, low-confidence classes. **(2)** Raise the input resolution back to 128×128; several of the classes bleeding into Bacterial spot are ones whose distinguishing features are fine-textured. **(3)** If it persisted after both, I would look at the decision threshold rather than the model — for a diagnostic tool you might deliberately accept lower precision on a class to protect recall elsewhere. What I would *not* do first is add capacity; nothing in the curves suggests the model is capacity-limited rather than training-limited.

Q: Why show the most confident misclassifications rather than a random sample? A: Because they are diagnostic and random ones mostly are not. An error the model was 99% sure about points at a systematic confusion — two classes it has genuinely conflated, or a mislabelled training image. An error it was 35% sure about is just the model saying it does not know, which tells you little you could act on. Sorting errors by confidence is a fast way to find the structural problems rather than the noise.

Phase 6 — Flask app

What was built

- `app/app.py` — three routes: `GET /` (upload page), `POST /predict`

(top-3 predictions as JSON), `GET /health` (liveness, 503 if the model failed to load). Model and class names load once at startup.

- `app/templates/index.html` — single page with drag-and-drop upload, image preview, confidence bars, and a domain-gap disclaimer.
- `app/static/style.css` — plain CSS, no framework.
- `app/README.md` — run instructions and the full error-handling table.

Tested end to end: server started, three real leaf images posted via `curl`, JSON verified, three UI screenshots captured with a headless browser, server stopped.

Why — design decisions

Why Flask rather than Django or FastAPI? The app is two meaningful routes. Django would bring an ORM, migrations and an admin panel that would all sit unused — its value shows up on database-backed applications with many models, which this is not. FastAPI would be the better choice for a pure JSON API with generated docs and async I/O, but this app renders an HTML page and does CPU-bound synchronous inference, so async buys nothing. Flask does exactly what is needed and nothing else, which also means a reviewer can read the whole server in one sitting.

Why load the model once at startup rather than per request? Loading a Keras model means reading the file, rebuilding the graph and allocating weights. Doing that inside the request handler would add that cost to every prediction and would hold multiple copies of the model in memory under concurrent load. Loading once at import

time means a request only runs a forward pass. The trade-off is slower startup and needing a restart to pick up a retrained model — both fine here, and it is the standard production pattern too. The failure mode to handle is that startup loading can fail, which is why the load is wrapped in a `try/except` and `/health` reports 503 rather than the app dying silently.

Why inference preprocessing must match training exactly — and how it bit me. This is the most valuable thing in this phase. The first implementation resized with Pillow's `Image.BILINEAR`, which looks like the obvious equivalent of `tf.image.resize(..., 'bilinear')`. It is not. **Pillow applies an antialiasing filter when downscaling**; `tf.image.resize` with the default `antialias=False` **does not** — it samples. Going $256 \times 256 \rightarrow 64 \times 64$ is a $4 \times$ reduction, where that distinction is severe.

Measured on three test leaves: **max per-pixel difference 0.19** on a 0–1 scale, and **the predicted class changed on 2 of 3 images**. A healthy leaf was confidently called Bacterial spot at 90%. There was no error, no warning, no crash — just quietly wrong answers, which is the worst kind of bug.

The fix was to use the identical TensorFlow op in the app, reproducing the training pipeline step for step including its intermediate round $\rightarrow$ `uint8` quantisation. After the fix: max difference **0.0157** (residual JPEG-decoder variation between PIL and `tf.io.decode_jpeg`) and **0 disagreements**; the same healthy leaf now scores 98.2% healthy.

Two habits came out of this: the app **imports** `IMAGE_SIZE` **from** `src/data_loader.py` rather than hardcoding it, so a retrain at a different resolution cannot desynchronise them; and class names come from `sorted(unique labels)` — the exact expression training used — because any other ordering would mislabel every prediction while looking entirely plausible.

Why return top-3 with confidences instead of just the argmax? Several of these diseases genuinely look alike, and a single label hides how close the call was. "Early blight 51%, Late blight 47%" is far more honest, and more actionable, than "Early blight" presented as certain. It also surfaces the model's failure modes to the user instead of hiding them — which matters especially for a model that is 76.76% accurate, not 99%.

Error handling

Every failure returns JSON with a helpful message and a correct status code — never a stack trace, and never internal detail:

Situation	Status	Behaviour
No file field	400	"No file was uploaded..."
Empty filename	400	"No file was selected."
Wrong extension	400	Names the rejected type
Empty file	400	"The uploaded file is empty."
Corrupt / not an image	400	Caught via <code>Image.load()</code> forcing a full decode
Over 8 MB	413	Flask <code>MAX_CONTENT_LENGTH</code>
Model not loaded	503	Tells the user to train first
Inference raised	500	Generic message; traceback to server log only

`debug=False` deliberately: the reloader would load the model twice, and the interactive debugger must never be exposed by a demo left running.

Likely interview questions

Q: What happens to this app under real production load? A: As written, it would not survive much.

`app.run()` is Flask's development server — single-process, and not built for concurrency. The first fix is to put it behind a WSGI server such as Gunicorn or Waitress with several workers. That immediately raises a memory question: each worker loads its own copy of the model, so N workers means N times the model's footprint — fine for a 1.4 MB model like this one, a real constraint for a 500 MB one, where the answer is a dedicated

inference service (TensorFlow Serving) that several lightweight web workers call. Beyond that: batching concurrent requests to amortise inference, a request-size limit (already present), rate limiting, and a real object store for uploads rather than holding them in memory. I would also add structured logging of prediction distributions, because the way you find out a model has gone stale in production is usually that its confidence distribution drifts.

Q: How would you know if the deployed model started performing badly? A: Not from accuracy, because production has no labels. The practical signals are indirect: monitor the **confidence distribution** — a model fed inputs unlike its training data tends to get less confident, or confidently wrong in new patterns — and the **predicted-class distribution**, since a sudden spike in one class usually means input drift rather than a real outbreak. Both are computable without ground truth. Beyond that I would sample a small fraction of real uploads for manual labelling to get a genuine accuracy estimate over time. For this model specifically the expected failure is the domain gap: users uploading field photos rather than lab shots, which is exactly the distribution shift the confidence monitor would catch first.

Q: Someone uploads a photo of a dog. What does your app do, and is that OK? A: It confidently returns a tomato disease, because a softmax over 10 classes must sum to 1 — the model has no way to say "none of these". That is a genuine limitation, not a bug I can fix with better error handling. The honest mitigations are: showing the top-3 with confidences so absurd outputs are at least visible; adding a confidence threshold below which the app says "not sure, is this a tomato leaf?"; or properly, training an out-of-distribution detector or adding a "not a leaf" class with negative examples. For a portfolio demo I chose the first two-thirds of that — surface the uncertainty, and state the scope in a disclaimer on the page — and I would flag OOD detection as required work before anything like this went in front of real users.

Elevator Pitch

For the "so, tell me about this project" opener.

I built an end-to-end crop disease classifier: it takes a photo of a tomato leaf and tells you which of ten diseases it has, or that it's healthy. The model is a small convolutional neural network — about 112,000 parameters, three conv blocks — trained from scratch on the PlantVillage tomato subset, roughly 18,000 images across 10 classes. It's served through a Flask app where you upload an image and get the top three predictions with confidence scores. On the held-out test set it reaches **76.8% accuracy with a macro-F1 of 0.74**, against a majority-class baseline of 29.5%. The whole pipeline is reproducible from the repository: the dataset download is scripted, the train/validation/test split is committed as CSVs, and a CI job smoke-tests that the model still builds.

What I think makes it worth talking about isn't the accuracy number — it's that the interesting problems weren't the machine learning. **I profiled a `tf.data` cache expecting a big speedup and measured 0.97×**, because the workload was compute-bound rather than I/O-bound; I could show the arithmetic — about 165 million multiply-accumulates per image — that accounted for essentially the whole epoch time. Worse, **adding that cache required reordering the pipeline, which silently inflated a shuffle buffer from 1 MB to 596 MB** — the line of code never changed, only what flowed into it — and that paged a 7.5 GB laptop into a standstill and cost a five-hour training run. I diagnosed it by watching CPU-seconds per wall-second rather than wall-clock per epoch, because a compute-bound process getting 1.5 of 6 cores isn't slow, it's blocked. And when I built the Flask app, **end-to-end testing caught that Pillow's bilinear resize antialiases while TensorFlow's doesn't** — a difference large enough to flip the predicted class on two of three test images, with no error raised anywhere.

So the honest summary is: a working, deployed, reproducible classifier whose reported numbers are a floor rather than a ceiling — it's undertrained at five epochs on 64×64 inputs because I had to fit the hardware I had — plus a written record of three genuine engineering failures and how each was diagnosed and fixed. The obvious next step is transfer learning with MobileNetV2 on a GPU, which would likely reach the mid-90s in a fraction of the time, because the model would no longer have to learn edge detectors from scratch on 12,000 images.

Appendix — Anticipated Questions

Every question from every phase, in one place, with a compressed answer. Full reasoning lives in the phase section named in each heading.

Project structure and tooling (Phase 1)

Why structure the project this way instead of one script? Separating data, source, models and app lets each change independently, and the layout is a trimmed *Cookiecutter Data Science* template so it's immediately familiar. One script is fine for a throwaway experiment; structure earns its keep once stages get re-run at different times.

What if you committed the dataset and trained model to Git? The repo balloons past a gigabyte, clones take minutes, GitHub rejects files over 100 MB, and because Git snapshots whole binaries, retraining a few times bloats history permanently — deleting later doesn't shrink it. Both artefacts are reproducible, so committing the code that makes them is enough. Alternatives: Git LFS or DVC.

Why bother with a virtual environment? Isolates this project's libraries so another project's NumPy can't break it, and it makes `requirements.txt` honest because the environment started empty.

Why gitignore .env and kaggle.json? `kaggle.json` is an API key tied to my account; secret-scanning bots find leaked keys within minutes, and deleting the file in a later commit doesn't help because it stays in history. The only fix after a leak is revocation — so don't commit it.

Dataset and EDA (Phase 2)

Why not the full 54,000-image dataset? Practically, 18k vs 54k is ~3× epoch time on a CPU, which decides how many experiments I can afford. More interestingly, the full set is partly an *easier* problem — much of it is telling apples from corn, solvable from leaf shape. One crop forces discrimination between diseases of the same species, which is the genuinely hard part.

You have a 14× class imbalance. How did you handle it? Measured it, then let it drive three decisions: quote the 29.5% majority baseline alongside any accuracy; stratify every split so the 373-image class isn't left to chance; report a confusion matrix and per-class recall rather than accuracy alone. I used inverse-frequency `class_weight` in training and deliberately did not resample first — establish a baseline, then fix what the confusion matrix shows.

What did the EDA change about your plan? Total image uniformity (every file 256×256 RGB) removed the aspect-ratio handling I'd expected to need; zero corrupt files removed defensive error handling from the loader; and the imbalance figure drove stratification plus confusion-matrix-based evaluation.

What's the dataset's biggest weakness? The domain gap: lab photos of single detached leaves on plain backgrounds. The test split shares that unrealistic distribution, so a strong score proves the pipeline works, not that the model is field-ready.

Preprocessing (Phase 3)

Why augment only training data? Training data teaches, so distorting it helps generalisation. Validation and test *measure*, and augmenting them would score the model on inputs no user submits and make results jitter run to run. Subtle trap: Keras preprocessing layers are no-ops at `training=False`, so I applied them explicitly in the input pipeline and wrote a test asserting two validation passes are bit-for-bit identical.

Why those augmentations and not others? Rule: it must change the image without ever changing the correct label, and model a variation that really occurs. Flips, ±10% rotation and ±10% zoom pass. Colour jitter fails — several of these diseases are identified *by* hue, so shifting it can make mosaic virus look like early blight while the label still says mosaic virus. Random cropping fails too: crop out the only lesion and you have a healthy leaf labelled diseased.

How do you prevent data leakage between splits? Split on file paths before any image loads, so one image lands in exactly one split; persist the split to CSV and commit it so it can't be silently reshuffled; fix `random_state`; and keep all preprocessing statistics constant (a fixed /255, not a mean computed from data). The subtler one I guard against is *selection* leakage — validation drove early stopping and checkpointing, so I report on test.

Why 128×128 rather than native 256×256? (later reduced to 64×64) Halving each dimension quarters the convolution work, which is what makes multiple experiments affordable on a CPU. Below 128 the fine speckling separating Septoria leaf spot from Target Spot starts to smear — and when hardware later forced 64×64, exactly that confusion appeared in the results.

Architecture and training (Phase 4)

112k parameters — isn't that far too small? The data sets the ceiling: 12,712 training images can be memorised by a multi-million-parameter model. It's also where `GlobalAveragePooling2D` pays off — `Flatten` would put ~1M parameters in a single dense layer. If the model *underfit*, I'd add a fourth conv block before touching the head; nothing in the curves suggests capacity is the limit rather than training time.

Validation loss 3.79 vs training loss 1.18 after one epoch — overfitting? Almost certainly not after 398 gradient steps. It's BatchNorm's train/inference mismatch: batch statistics during training, poorly-converged moving averages at inference. The distinguishing test is to keep training — and this resolved exactly as predicted, with val loss collapsing from 2.96 to 0.83 at epoch 4.

Why a one-epoch benchmark instead of just training? Turns an unknown into a number (8.6 min/epoch → 4.3 hours for 30), and it's the cheapest smoke test for a broken pipeline: loss near $\ln(10)=2.30$ would mean mislabelled data or a bad learning rate, discovered in minutes rather than hours.

Why `class_weight` over oversampling or focal loss? Costs nothing — no extra data, no longer epochs, one argument to `fit()`, so it's trivial to ablate. Oversampling would duplicate the 373-image class ~14× and invite memorising those specific images. Focal loss is built for ~1000:1 imbalance and adds a hyperparameter; at 14.4× it's a heavier tool than the problem needs.

Why early-stop on `val_loss` not `val_accuracy`? With 14.4× imbalance, accuracy can rise while the model gets worse at rare classes. Loss is computed on probabilities, so it detects falling confidence before the arg-max flips, and since `class_weight` applies to the loss, the monitored quantity is the class-balanced objective actually being optimised. Accuracy is also a step function — poor signal for a patience rule.

Walk me through the 5-hour training run that had to be killed. Adding a cache required moving shuffle after decode, which changed the buffer from 1 MB of paths to 596 MB of images without the line itself changing. The machine paged: parallelism fell 7.4× → 1.5×, 2.7 GB pagefile, ~11 of 30 epochs in 5h21m. Diagnosed via CPU-seconds per wall-second, not wall-clock. Fixed by bounding the buffer at 2,048, dropping the cache (already measured at 0.97×), and adding a startup RAM guard.

Evaluation (Phase 5)

76.76% accuracy — is that good? Against a 29.5% baseline it's about +47 points, but it isn't the architecture's ceiling: the run hit a 5-epoch cap with validation loss still falling, on 64×64 inputs chosen because larger configs wouldn't complete on my hardware. Published PlantVillage models reach the high 90s; the gap is explained by resolution, training length and no transfer learning — all nameable, not hand-waved.

Why macro-F1 as well as accuracy? Accuracy hides per-class failure under imbalance. Macro-F1 gives every class an equal vote. Here they're close (0.745 vs 0.773), and that closeness is itself the result — the class weighting worked.

How do you know test data didn't leak into training? Path-level split made once before loading; split committed to CSV so it can't reshuffle; fixed seed; constant preprocessing statistics. Plus reporting on test rather than validation, since validation drove early stopping and checkpointing.

The model over-predicts Bacterial spot — what would you do? It's the class the model retreats to when uncertain (0.99 recall, 0.54 precision). In order: train longer (it hit the epoch cap while improving), raise

resolution back to 128×128 (the classes bleeding into it are fine-textured), then consider decision thresholds. Adding capacity would be last — the evidence says training-limited, not capacity-limited.

Why show the most confident misclassifications, not random ones? A 99%-confident error points at a systematic confusion or a mislabelled image; a 35%-confident one is just the model admitting uncertainty. Sorting by confidence finds structure rather than noise.

Deployment (Phase 6)

What happens under real production load? `app.run()` is a single-process dev server. First fix is Gunicorn/Waitress with several workers — which raises memory, since each worker loads its own model copy (fine at 1.4 MB, a real constraint at 500 MB, where the answer is TensorFlow Serving). Then request batching, rate limiting, size limits, and structured logging of prediction distributions.

How would you know if the deployed model degraded? Production has no labels, so watch indirect signals: the **confidence distribution** (drifts when inputs stop resembling training data) and the **predicted-class distribution** (a sudden spike usually means input drift, not an outbreak). Both need no ground truth. Then sample a fraction of real uploads for manual labelling.

Someone uploads a photo of a dog — what happens? It confidently returns a tomato disease, because a softmax over 10 classes must sum to 1. That's a genuine limitation, not a fixable bug. Mitigations: show top-3 confidences, add a confidence threshold, or properly train an OOD detector or a "not a leaf" class. I did the first two-thirds and would flag OOD detection as required before real users.

Why does inference preprocessing have to match training exactly? Because mismatches fail silently. Mine did: Pillow's BILINEAR antialiases when downscaling, `tf.image.resize` (default `antialias=False`) does not. At 256→64 that produced a 0.19 max per-pixel difference and changed the predicted class on 2 of 3 images — no error, just worse answers. Fixed by using identical TensorFlow ops and importing `IMAGE_SIZE` from the training module instead of hardcoding it.